potentially cause undefined behavior if it's accessed before being assigned a new value:

```
X x1;                                   <-- x1.a has an indeterminate value
```

If, on the other hand, you initialize your instance of `X` by explicitly invoking the default constructor, then `a` is initialized to zero:

```
X x2=X();                               <-- x2.a==0
```

This bizarre property also extends to base classes and members. If your class has a compiler-generated default constructor and any of your data members and base classes also have a compiler-generated default constructor, data members of those bases and members that are built-in types are also either left with an indeterminate value or initialized to zero, depending on whether or not the outer class has its default constructor explicitly invoked.

Although this rule is confusing and potentially error prone, it does have its uses, and if you write the default constructor yourself, you lose this property; either data members like `a` are always initialized (because you specify a value or explicitly default construct) or always uninitialized (because you don't):

```
X::X():a(){}        <-- a==0 always
X::X():a(42){}      <-- a==42 always
X::X(){}            <-- (1)
```

If you omit the initialization of `a` from the constructor of `X` as in the third example (1), then `a` is left uninitialized for nonstatic instances of `X` and initialized to zero for instances of `X` with static storage duration.

Under normal circumstances, if you write any other constructor manually, the compiler will no longer generate the default constructor for you, so if you want one you have to write it, which means you lose this bizarre initialization property. However, by explicitly declaring the constructor as defaulted, you can force the compiler to generate the default constructor for you, and this property is retained: